

Experiment 1: Basic Input/Output and Conditional Operations

1a. Basic Arithmetic Calculator

Aim: Develop a program to read two numbers from the keyboard and perform basic arithmetic operations (Addition, Subtraction, Multiplication, Division) based on user choice.

Algorithm/Approach:

1. Read two integers from the user.
2. Read the operation choice (1-4).
3. Use if-elif-else to perform the selected operation.
4. Display the result; handle division by zero.

Basic Arithmetic Calculator

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
choice = int(input("Enter choice (1-Add, 2-Subtract, 3-Multiply, 4-Divide): "))

if choice == 1:
    result = num1 + num2
    print(f"Addition: {num1} + {num2} = {result}")
elif choice == 2:
    result = num1 - num2
    print(f"Subtraction: {num1} - {num2} = {result}")
elif choice == 3:
    result = num1 * num2
    print(f"Multiplication: {num1} * {num2} = {result}")
elif choice == 4:
    if num2 != 0:
        result = num1 / num2
        print(f"Division: {num1} / {num2} = {result}")
    else:
        print("Error: Division by zero!")
else:
    print("Invalid choice!")
```

Sample Input/Output:



Enter first number: 10

Enter second number: 5

Enter choice (1-Add, 2-Subtract, 3-Multiply, 4-Divide): 1

Addition: $10 + 5 = 15$

Theory/Notes: Uses conditional statements for menu-driven input. Ensure type conversion with `int()` for numeric inputs.

1b. Senior Citizen Checker

Aim: Develop a program to read the name and year of birth of a person and display whether the person is a senior citizen (age ≥ 60).

Algorithm/Approach:

1. Read name and birth year as strings/integers.
2. Compute age using current year (2025).
3. If age ≥ 60 , print "Senior Citizen"; else, "Not a Senior Citizen".

Senior Citizen Checker

```
from datetime import datetime

current_year = datetime.now().year

print("Current year:", current_year)

name = input("Enter the person's name: ")

year_of_birth = int(input("Enter the year of birth: "))

age = current_year - year_of_birth

if age >= 60:

    print(f"{name} is a senior citizen (Age: {age}).")

else:

    print(f"{name} is not a senior citizen (Age: {age}).")
```

Sample Input/Output:

Enter name: Alice

Enter year of birth: 1950

Alice is a Senior Citizen (Age: 75).

Experiment 2: Sequences and List Operations

2a. Fibonacci Sequence Generator

Aim: Develop a program to generate a Fibonacci sequence of length N, read from the console.

Algorithm/Approach:

1. Read integer N.
2. Initialize first two terms (0, 1).
3. Generate subsequent terms using a loop.
4. Print the sequence.

Fibonacci Sequence Generator

```
a,b=0,1
while a<=10:
    print(a,end=",")
    a, b = b, a + b
```

Sample Input/Output:

Enter length N: 8

Fibonacci sequence of length 8: [0, 1, 1, 2, 3, 5, 8, 13]

Theory/Notes: Fibonacci: Each term is the sum of the two preceding ones. Handle edge cases (N=0,1).

2b. List Operations Demo

Aim: Write a program to create a list and perform operations: Insert, Remove, Append, Display length, Pop, Clear.

Algorithm/Approach:

1. Create an empty list.
2. Prompt user for operations and elements.
3. Use list methods: insert(index, elem), remove(elem), append(elem), len(), pop(index), clear().

List Operations Demo

```
my_list = []
```

Insert

```
my_list.insert(0, 10)
```

```
print("After insert: ", my_list)
```

Append

```
my_list.append(20)
```

```
print("After append: ", my_list)
```

Remove

```
my_list.remove(10)
```

```
print("After remove: ", my_list)
```

Length

```
print("Length: ", len(my_list))
```

Pop

```
popped = my_list.pop()
```

```
print("Popped: ", popped, "List now: ", my_list)
```



MOODLAKATTE
INSTITUTE OF TECHNOLOGY

(Affiliated to VTU, Belagavi & Approved by AICTE, New Delhi)
Moodlakatte - 576217, Kundapura Taluk, Udupi District, Karnataka

Clear

```
my_list.clear()
```

```
print("After clear: ", my_list)
```

Sample Input/Output:

After insert: [10]

After append: [10, 20]

After remove: [20]

Length: 1

Popped: 20 List now: []

After clear: []

Theory/Notes: Lists are mutable sequences. `pop()` removes and returns the last element by default.

Experiment 3: Statistical Computations and String Processing

3a. Mean, Variance, and Standard Deviation

Aim: Read N numbers, create a list, and print mean, variance, and standard deviation.

Algorithm/Approach:

1. Read N, then N numbers into a list.
2. Compute mean = sum/len.
3. Variance = sum((x - mean)²)/len.
4. Std Dev = sqrt(variance).
5. Use math.sqrt for square root.

Statistical Computations

import math

N = int(input("Enter N: "))

numbers = []

for _ in range(N):

numbers.append(float(input("Enter number: ")))

mean = sum(numbers) / N

variance = sum((x - mean) ** 2 for x in numbers) / N

std_dev = math.sqrt(variance)

print(f"Mean: {mean:.2f}")

print(f"Variance: {variance:.2f}")

print(f"Standard Deviation: {std_dev:.2f}")

Sample Input/Output:

Enter N: 3



Enter number: 1

Enter number: 2

Enter number: 3

Mean: 2.00

Variance: 0.67

Standard Deviation: 0.82

Theory/Notes: Population statistics (divide by N). For sample, divide by N-1.

3b. Digit Frequency Counter

Aim: Read a multi-digit number as characters and print the frequency of each digit.

Algorithm/Approach:

1. Read number as string.
2. Use dictionary to count occurrences of each digit.
3. Print frequencies.

Digit Frequency

```
num_str = input("Enter multi-digit number: ")
freq = {}
for char in num_str:
    if char.isdigit():
        freq[char] = freq.get(char, 0) + 1

print("Digit Frequencies:")
for digit, count in sorted(freq.items()):
    print(f"{digit}: {count}")
```

Sample Input/Output:

Enter multi-digit number: 1223

Digit Frequencies:

1: 1

2: 2

3: 1

Theory/Notes: Treat as string to access individual chars. Use collections.Counter for efficiency.

Experiment 4: File Word Frequency Analysis

Aim: Develop a program to print the 10 most frequently appearing words in a file using a dictionary.

Algorithm/Approach:

1. Read file content, split into words (lowercase, remove punctuation).
2. Use dictionary to count frequencies.
3. Sort by frequency (descending), slice top 10.

Top 10 Words in File

```
from collections import Counter
import string
```

```
filename = "sample.txt" # Assume sample.txt exists
```

```
with open(filename, 'r') as file:
```

```
    = file.read().lower()
```

```
    words = [word.strip(string.punctuation) for word in .split() if
word.strip(string.punctuation)]
```

```
word_freq = Counter(words)
```

```
top_10 = word_freq.most_common(10)
```

```
print("Top 10 Most Frequent Words:")
```

```
for word, freq in top_10:
```

```
    print(f"{word}: {freq}")
```

Sample Input/Output (Assuming sample.txt: "Hello world. Hello again world."):

Top 10 Most Frequent Words:

hello: 2

world: 2

again: 1

Theory/Notes: Use Counter for simplicity. Sort with `most_common()`. Handle punctuation with `string.punctuation`.

Experiment 5: Sorting Student Marks with Bubble Sort

Aim: Read 6 subject marks, sort from highest to lowest using Bubble Sort, and display.

Algorithm/Approach:

1. Read 6 marks into a list.
2. Implement Bubble Sort (descending).
3. Print sorted marks.

Bubble Sort for Marks (Descending)

```
marks = []  
for _ in range(6):  
    marks.append(int(input("Enter mark: ")))  
  
# Bubble Sort Descending  
n = len(marks)  
for i in range(n):  
    for j in range(0, n - i - 1):  
        if marks[j] < marks[j + 1]:  
            marks[j], marks[j + 1] = marks[j + 1], marks[j]  
  
print("Sorted Marks (Highest to Lowest):", marks)
```

Sample Input/Output:

```
Enter mark: 85  
Enter mark: 92  
Enter mark: 78  
Enter mark: 95  
Enter mark: 88  
Enter mark: 70
```

Sorted Marks (Highest to Lowest): [95, 92, 88, 85, 78, 70]

Theory/Notes: Bubble Sort: Repeatedly swap adjacent if out of order. $O(n^2)$ time.

Experiment 6: File Sorting

Aim: Sort the contents of a file and write to a new file.

Algorithm/Approach:

1. Read lines from input file.
2. Strip whitespace, sort the list.
3. Write sorted lines to output file.

Sort File Contents

```
input_file = "input.txt"
```

```
output_file = "sorted_output.txt"
```

```
with open(input_file, 'r') as f:
```

```
    lines = [line.strip() for line in f.readlines() if line.strip()]
```

```
lines.sort()
```

```
with open(output_file, 'w') as f:
```

```
    for line in lines:
```

```
        f.write(line + '\n')
```

```
print("Sorted contents written to", output_file)
```

Sample Input/Output (input.txt: "banana\napple\ncherry"):

- sorted_output.txt: apple\nbanana\ncherry

Theory/Notes: Use strip() to clean lines. sort() is case-sensitive; use key=str.lower for insensitive.

Experiment 7: Exception Handling in Division Function

Aim: Develop DivExp function with assertion for $a > 0$ and exception for $b = 0$.

Algorithm/Approach:

1. Define function with params a, b.
2. Assert $a > 0$.
3. If $b == 0$, raise ValueError.
4. Return a / b .
5. Main: Read a, b; call function.

DivExp with Exceptions

```
def DivExp(a, b):  
    assert a > 0, "a must be positive"  
    if b == 0:  
        raise ValueError("Division by zero!")  
    return a / b
```

```
a = float(input("Enter a (>0): "))  
b = float(input("Enter b: "))  
try:  
    result = DivExp(a, b)  
    print(f"Result: {result}")  
except AssertionError as e:  
    print(f"Assertion Error: {e}")  
except ValueError as e:  
    print(f"Value Error: {e}")
```

Sample Input/Output:

Enter a (>0): 10

Enter b: 2

Result: 5.0

Enter a (>0): 10

Enter b: 0

Value Error: Division by zero!

Experiment 8: Complex Number Addition

Aim: Define a Complex class, function to add two complexes, and sum N complexes.

Algorithm/Approach:

1. Class Complex with real, imag; `__init__`, `__str__`.
2. Function `add_complex(c1, c2)` returns new Complex.
3. Main: Read N, read N complexes, sum iteratively.

Complex Number Class

class Complex:

def `__init__`(self, real, imag):

self.real = real

self.imag = imag

def `__str__`(self):

return f"{self.real} + {self.imag}i"

def add(self, other):

return Complex(self.real + other.real, self.imag + other.imag)

def add_complex(c1, c2):

return c1.add(c2)

N = int(input("Enter N (>=2): "))

total = Complex(0, 0)

for _ in range(N):

real = float(input("Real: "))

imag = float(input("Imag: "))

c = Complex(real, imag)

```
total = total.add(c)
```

```
print("Sum:", total)
```

Sample Input/Output (N=2, c1:1 2, c2:3 4):

Sum: 4.0 + 6.0i

Theory/Notes: Operator overloading possible with `__add__`. Here, use method for clarity.

Experiment 9: Analysis Tool

Aim: Analyze a paragraph: word frequency, longest word, number of sentences, etc.

Algorithm/Approach:

1. Read paragraph.
2. Split words, count freq with Counter.
3. Sentences: split by .!?.
4. Longest word: max(len) among words.

Analysis Tool

```
from collections import Counter
```

```
import string
```

```
paragraph = input("Enter paragraph: ")
```

```
words = [word.strip(string.punctuation).lower() for word in paragraph.split()]
```

```
sentences = [s.strip() for s in paragraph.split('.') if s.strip()] # Simple sentence split
```

```
word_freq = Counter(words)
```

```
longest_word = max(words, key=len) if words else ""
```

```
print("Word Frequencies:", dict(word_freq))
```

```
print("Number of Sentences:", len(sentences))
```

```
print("Longest Word:", longest_word, f"(Length: {len(longest_word)})")
```

Sample Input/Output:

Enter paragraph: Hello world. This is a test.

Word Frequencies: {'hello': 1, 'world': 1, 'this': 1, 'is': 1, 'a': 1, 'test': 1}

Number of Sentences: 2

Longest Word: hello (Length: 5)

Theory/Notes: Basic analysis; enhance with regex for better sentence detection.

Experiment 10: Data Summary Generator

Aim: Read a CSV file, convert to dictionary, allow queries: max, min, average by column.

Algorithm/Approach:

1. Read CSV with csv module into list of dicts.
2. For query: Select column, compute max/min/avg using loops or statistics.

Data Summary Generator

```
import csv
from statistics import mean

filename = "data.csv" # Assume data.csv: col1,col2\n1,2\n3,4
data = []
with open(filename, 'r') as f:
    reader = csv.DictReader(f)
    for row in reader:
        data.append(row)

col = input("Enter column name: ")
values = [float(row[col]) for row in data if col in row]
if values:
    print(f"Max: {max(values)}")
    print(f"Min: {min(values)}")
    print(f"Average: {mean(values):.2f}")
else:
    print("Column not found!")
```

Sample Input/Output (data.csv as above, col='col1'):

Enter column name: col1

Max: 3.0

Min: 1.0

Average: 2.00

Experiment 11: Student Grade Tracker

Aim: Accept multiple students' names and marks, store in list of tuples/dicts, display summaries.

Algorithm/Approach:

1. Read num students, then name/marks for each (dict: {'name': , 'marks': [list]}).
2. Compute avg per student, overall avg, topper.

```
students = []
N = int(input("Enter number of students: "))
for _ in range(N):
    name = input("Name: ")
    marks = list(map(int, input("Marks (space-separated): ").split()))
    students.append({'name': name, 'marks': marks, 'avg': sum(marks)/len(marks)})

# Summaries
print("\nStudent Averages:")
for s in students:
    print(f"{s['name']}: {s['avg']:.2f}")

overall_avg = mean([s['avg'] for s in students])
topper = max(students, key=lambda x: x['avg'])
print(f"Overall Average: {overall_avg:.2f}")
print(f"Topper: {topper['name']} ({topper['avg']:.2f})")
```

Sample Input/Output (N=2, Alice:90 85, Bob:95 92):

Student Averages:

Alice: 87.50

Bob: 93.50

Overall Average: 90.50

Topper: Bob (93.50)

Theory/Notes: Use list of dicts for flexibility. Handle variable num marks.

Experiment 12: Recursive Directory Lister

Aim: Display contents of a folder recursively (sub-folders and files with name and type).

Algorithm/Approach:

1. Use os module.
2. Define recursive function to walk dir, print files/folders.

Recursive Directory Lister

import os

def list_directory(path):

for item in os.listdir(path):

 item_path = os.path.join(path, item)

if os.path.isdir(item_path):

print(f"Folder: {item}")

 list_directory(item_path) *# Recurse*

else:

print(f"File: {item}")

path = input("Enter directory path: ")

list_directory(path)

Sample Input/Output (path='./test', contents: file1.txt, subdir/file2.py):

Enter directory path: ./test

File: file1.txt



Folder: subdir

File: file2.py

Theory/Notes: os.walk() alternative for non-recursive. Handle permissions with try-except.